Project Roadmap: AI-Driven XR Magnification and Checkpoint Assistance

MagXR Project

September 2026

1 Project Roadmap Overview

The development workflow is organized into five sequential phases:

Phase 1: VR Foundation (Unity & Quest 3)
↓
Phase 2: Local Audio Pipeline (Microphone & Transcription)
↓
Phase 3: Agentic Reasoning & Confirmation (LangGraph + Gemini)
↓
Phase 4: Unity–AI Bridge (WebSockets & Scene Controls)
↓
Phase 5: User Evaluation (Trials, NASA-TLX, SUS)

—

2 Recommended AI Platform & Models

- **Orchestration Framework: LangGraph** (paired with `langchain-google-genai`). LangGraph provides state-machine primitives—specifically state persistence and the `interrupt()` function—essential for human-in-the-loop workflows where the user visually inspects and confirms a parsed voice command in VR prior to tool execution.
- **Model Selection: Google Gemini 2.5 Flash** (`gemini-2.5-flash`) accessed via the **Google AI Studio Free Tier**.
  - *Free Tier Suitability:* Gemini 2.5 Flash offers high requests-per-minute (RPM) limits and low inference latency on the Google AI Studio free tier. It natively handles multimodal inputs (such as direct audio bytes) and supports structured outputs via Pydantic (`.with_structured_output()`).
  - *Resource Efficiency:* Running audio transcription and schema extraction in the cloud via Gemini preserves the PC's GPU VRAM solely for Unity XR rendering and Meta Quest Link video streaming.

—

3 Step-by-Step Implementation Process & Tooling

3.1 Step 1: VR Environment & Magnification Setup (Unity Engine)

Construct the minimalist 3D environment containing the high-resolution 2D map display and the target colored cubes.

- **Tools Required:**
  - Unity Engine (LTS version, Universal Render Pipeline / URP recommended)
  - Meta XR All-in-One SDK / XR Interaction Toolkit (configured for Meta Quest 3 via Quest Link)
- **Action Items:**
  1. Implement the three stereoscopic magnification configurations:
     - **Selective Zoom:** A virtual loupe mesh rendered using a secondary camera and a target `RenderTexture`.
     - **Image Zoom:** Magnification applied specifically to the 2D map plane.
     - **Full-View Zoom:** Magnification altering the global camera view frustum.
  2. Develop a **Spatial Checkpoint Manager** capable of serializing and restoring view transforms (camera position, magnification level, and target bounds).
  3. Construct a floating **World-Space Confirmation Canvas** parented to the controller or loupe to present parsed voice commands to the user.

3.2 Step 2: Voice Capture & Audio Preprocessing (Python)

Capture speech commands directly from the Meta Quest 3 headset microphone over Quest Link.

- **Tools Required:**
  - Python 3.10+
  - `sounddevice`, `numpy`, `scipy`
  - **Oculus Virtual Audio Device** (configured as default recording device in Windows Sound Settings)
- **Action Items:**
  1. Use `sounddevice` to stream 16-bit PCM audio (mono, 16 kHz) directly into an in-memory byte buffer (`io.BytesIO`).
  2. Implement an input RMS volume/peak checker prior to recording to confirm that audio is streaming from the headset microphone.

3.3 Step 3: Transcription & Prompt Parsing (Gemini Free Model)

Convert raw audio into text and extract structured magnification parameters.

- **Tools Required:**
  - `google-genai` / `langchain-google-genai`
  - `pydantic`
  - Free API Key from Google AI Studio (`GEMINI_API_KEY`)
- **Action Items:**
  1. Transcribe the audio buffer using `gemini-2.5-flash` via direct inline byte transfer, or utilize local `faster-whisper` (`base` model) if minimizing network overhead.
  2. Define a validated Pydantic schema (`MagnificationPrompt`) capturing:
     - `direction`: `Literal["in", "out", "reset"]`
     - `zoom_factor`: Float bound between 1.0 and 10.0

- `zooming_speed`: `Literal["slow", "normal", "fast"]`
- `target_object`: Target scene identifier or string name

3. Enforce structured extraction using `llm.with_structured_output(MagnificationPrompt)`.

3.4 Step 4: Agent Orchestration & Human Confirmation (LangGraph)

Manage conversational state, human confirmation breakpoints, and tool execution.

- **Tools Required:**
  - `langgraph`
  - `langgraph.checkpoint.memory.MemorySaver`
  - `langchain_core.tools`
- **Action Items:**
  1. Construct a LangGraph state graph equipped with an `interrupt()` node that pauses execution and emits the parsed instruction to Unity.
  2. Resume the graph when the user triggers confirmation in the headset using `Command(resume={"confirme True})`.
  3. Bind confirmed outputs to custom tool nodes: `set_magnifier_zoom`, `move_magnifier_to_target`, `create_checkpoint`, and `reset_magnification`.

3.5 Step 5: Network Communication Bridge (WebSockets)

Dispatch parsed instructions to Unity and handle real-time scene updates.

- **Tools Required:**
  - `websockets` (Python asynchronous library)
  - `NativeWebSocket` or `websocket-sharp` (Unity C# package)
- **Action Items:**
  1. Host a lightweight WebSocket server on `ws://127.0.0.1:8080`.
  2. Serialize tool execution parameters into standardized JSON payloads (e.g., {`"action": "SET_ZOOM", "factor": 3.0, "duration": 0.4`}).
  3. Receive payloads in Unity C# (`MagnifierController.cs`) and apply smooth view transitions via `Mathf.SmoothDamp` or `Vector3.Lerp`.

3.6 Step 6: Empirical Evaluation & Data Logging

Execute the $3 \times 2 \times 2 \times 2$ within-subjects experimental design across at least 20 participants.

- **Tools Required:**
  - Custom Unity C# CSV Logger
  - NASA Task Load Index (NASA-TLX) assessment forms
  - System Usability Scale (SUS) questionnaire and structured interview protocols
- **Action Items:**
  1. Automatically record trial performance metrics: Task Completion Time (TCT), Error Rate (incorrect colors or overlooked targets), and navigation trajectories.

2. Statistically evaluate variations across magnification types, the presence/absence of AI assistance, and spatial checkpoint availability.